LINMA2710 - Scientific Computing
Graphics processing unit (GPU)

P.-A. Absil and B. Legat

☐ Full Width Mode ☐ Present Mode

Table of Contents

Sources

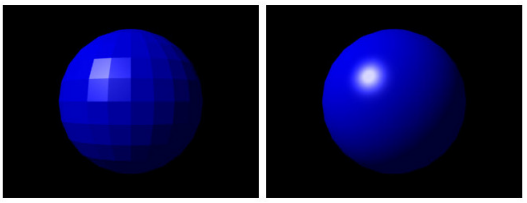
- OpenCL.jl
- HandsOnOpenCL

Context

- Most *dedicated* GPUs produced by NVIDIA and AMD
- *Integrated* GPUs by intel. used in laptops to reduce power consumption
- Designed for 3D rendering through ones of the APIs : Microsoft DirectX, OpenGL, WebGL, WebGPU, Vulkan. or Apple's Metal
- Illustration on the right is from Charge's film, it show how 3D modeling work.

General-Purpose computing on GPU (GPGPU)

Also known as *compute shader* as they abuses the programmable shading of GPUs by treating the data as texture maps.

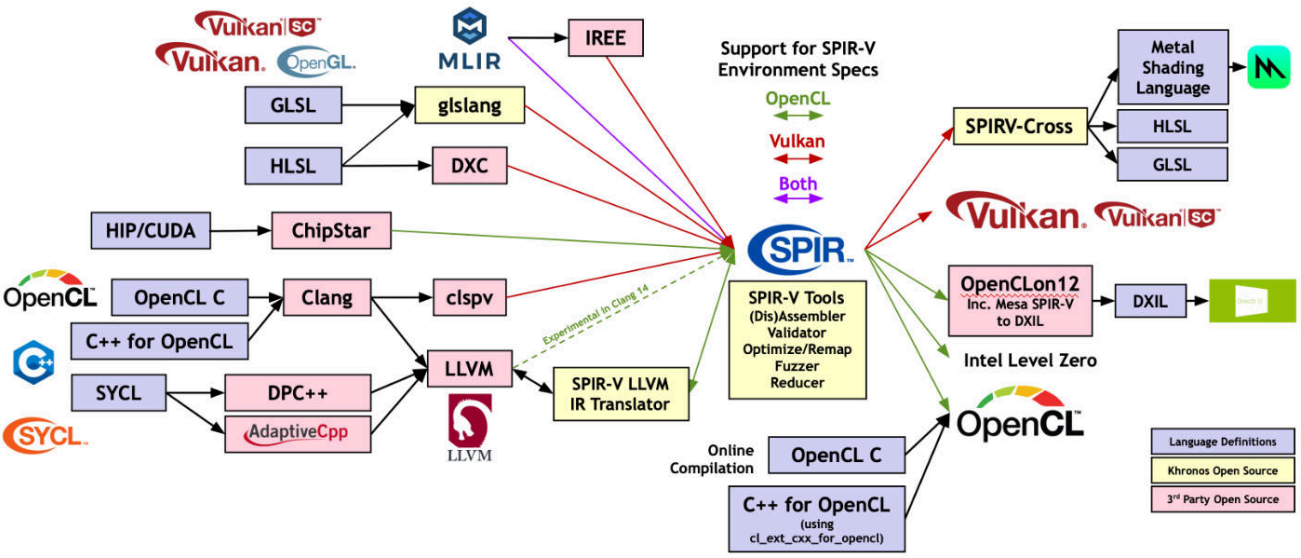


Hardware-specific

Common interface

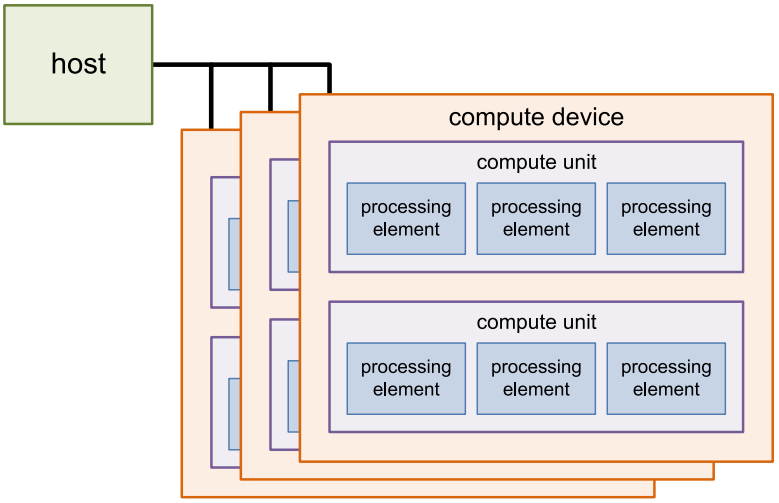
Standard Portable Intermediate Representation (SPIR)

Similar to LLVM IR : Intermediate representation for accelerated computation.



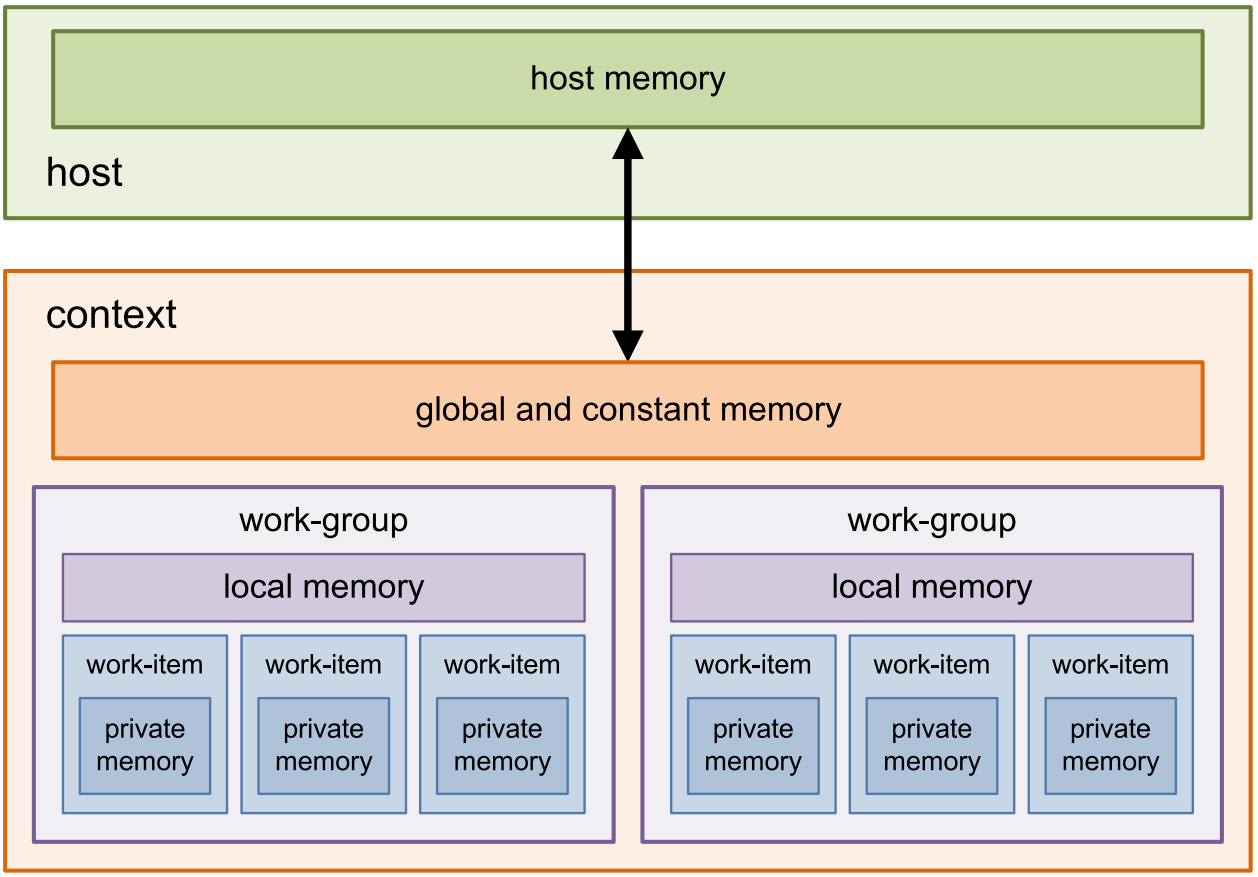
Hierarchy

- CPUs:
  - All CPUs part of same device
  - 1 Compute Unit per core
  - Number of processing elements equal to SIMD width
- GPUs:
  - One device per GPU



compute device	compute unit	processing element
`get_global_id`	`get_group_id`	`get_local_id`
`get_global_size`	`get_num_groups`	`get_local_size`

Memory



OpenCL Platforms and Devices

- Platforms are OpenGL implementations, listed in `/etc/OpenCL/vendors`
- Devices are actual CPUs/GPUs
- ICD allows to change platform at runtime.

```julia
OpenCL.versioninfo()
```

```
OpenCL.jl version 0.10.2

Toolchain:
 - Julia v1.11.4
 - OpenCL_jll v2024.5.8+1

Available platforms: 1
 - Portable Computing Language
   OpenCL 3.0, PoCL 6.0  Linux, Release, RELOC, SPIR-V, LLVM 16.0.6jl, SLEEF, DISTRO, POCL_DEBUG
   • cpu-haswell-AMD EPYC 7763 64-Core Processor (usm, fp16, fp64, il)
```

See also `clinfo` command line tool and `examples/OpenCL/common/device_info.c`.

Important stats

- Platform
  - name: Portable Computing Language
  - profile: FULL_PROFILE
  - vendor: The pocl project
  - version: PoCL 6.0 Linux, Release, RELOC, SPIR-V, LLVM 16.0.6jl, SLEEF, DISTRO, POCL_DEBUG
- Device
  - name: cpu-haswell-AMD EPYC 7763 64-Core Processor
  - type: cpu

clGetDeviceInfo	Value
CL_DEVICE_GLOBAL_MEM_SIZE	13.62 GiB
CL_DEVICE_MAX_COMPUTE_UNITS	4
CL_DEVICE_LOCAL_MEM_SIZE	512.00 KiB
CL_DEVICE_MAX_WORK_GROUP_SIZE	4096
CL_DEVICE_NATIVE_VECTOR_WIDTH_HALF	16
CL_DEVICE_NATIVE_VECTOR_WIDTH_FLOAT	8
CL_DEVICE_NATIVE_VECTOR_WIDTH_DOUBLE	4
CL_DEVICE_MAX_CLOCK_FREQUENCY	2445 MHz
CL_DEVICE_PROFILING_TIMER_RESOLUTION	1.000 ns

Portable Computing Language

cpu-haswell-AMD EPYC 7763 64-Core Processor

Vectorized sum

```
__kernel void vadd(
    __global const float *a,
    __global const float *b,
    __global float *c,
    int verbose) {
  int i = get_global_id(0);
  c[i] = a[i] + b[i];
```

vadd_size = 512

vadd_verbose = 0

```
5 warnings generated.
CL_KERNEL_WORK_GROUP_SIZE                    | 4096
CL_KERNEL_COMPILE_WORK_GROUP_SIZE            | (0, 0, 0)
CL_KERNEL_LOCAL_MEM_SIZE                     | 0 bytes
CL_KERNEL_PRIVATE_MEM_SIZE                   | 0 bytes
CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE | 8
Send command from host to device   | 1.192 µs
Including data transfer            | 300.733 ms
Execution of kernel                | 34.445 µs
```

Portable Computing Language

cpu-haswell-AMD EPYC 7763 64-Core Processor

vadd (generic function with 1 method)

Mandelbrot

```
__kernel void mandelbrot(__global float2 *q,
    __global ushort *output, ushort const maxit) {

  int gid = get_global_id(0), it;
  float tmp, real = 0, imag = 0;
  output[gid] = 0;
  for(it = 0; it < maxit; it++) {
    tmp = real * real - imag * imag + q[gid].x;
    imag = 2 * real * imag + q[gid].y;
    real = tmp;
    if (real * real + imag * imag > 4.0f)
        output[gid] = it;
```

mandel_size = 512

maxiter = 100

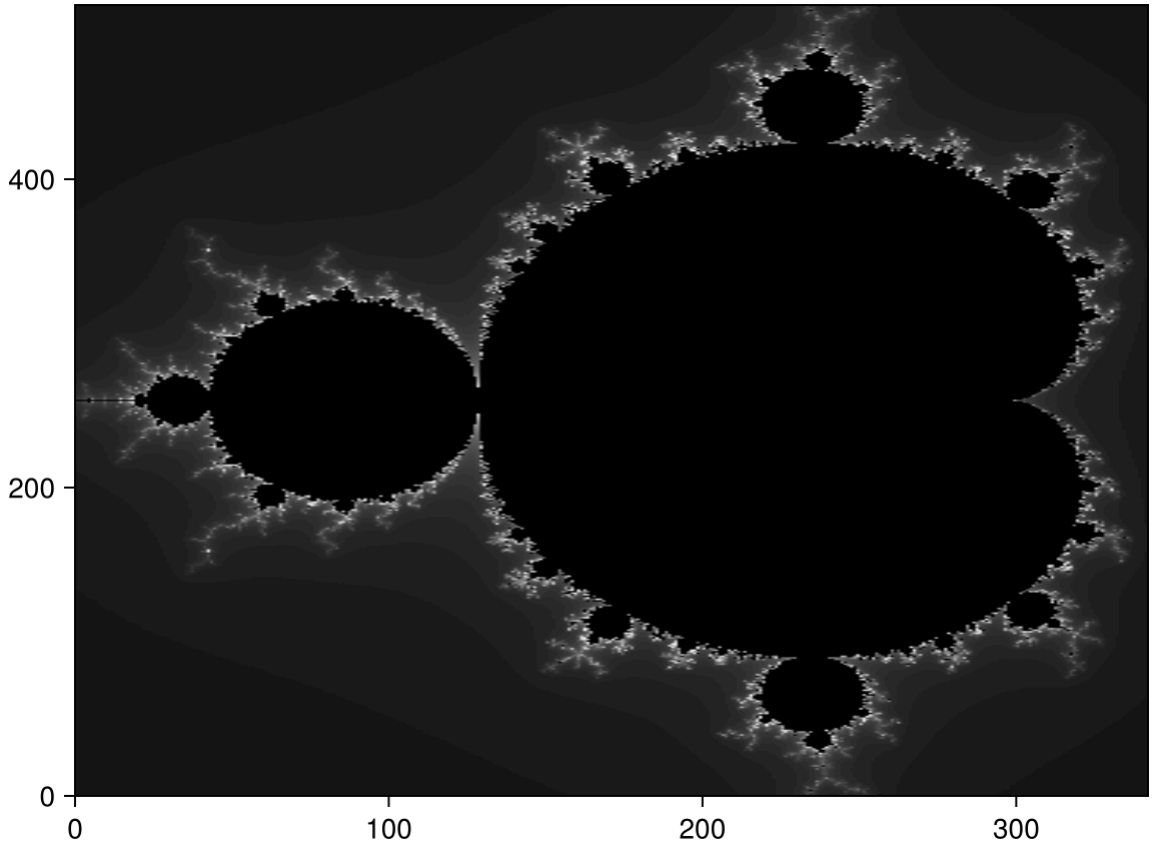
```
q = [ComplexF32(r,i) for i=1:-(2.0/mandel_size):-1, r=-1.5:(3.0/mandel_size):0.5];
```

```
mandel_image = mandel(q, maxiter, mandel_device);
```

```
CL_KERNEL_WORK_GROUP_SIZE                    | 4096
CL_KERNEL_COMPILE_WORK_GROUP_SIZE            | (0, 0, 0)
CL_KERNEL_LOCAL_MEM_SIZE                     | 0 bytes
CL_KERNEL_PRIVATE_MEM_SIZE                   | 0 bytes
CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE | 8
Send command from host to device   | 1.142 µs
Including data transfer            | 45.706 ms
Execution of kernel                | 13.845 ms
```

Portable Computing Language

cpu-haswell-AMD EPYC 7763 64-Core Processor




```
mandel (generic function with 1 method)
```

```julia
function mandel(q::Array{ComplexF32}, maxiter::Int64, device)
    cl.device!(device)
    q = CLArray(q)
    o = CLArray{Cushort}(undef, size(q))

    prg = cl.Program(; source = mandel_source.code) |> cl.build!
    k = cl.Kernel(prg, "mandelbrot")

    timed_clcall(k, Tuple{Ptr{ComplexF32}, Ptr{Cushort}, Cushort},
           q, o, maxiter; global_size=length(q))

    return Array(o)
end
```

Compute π

NaN

```
mypi()
```

```
CL_KERNEL_WORK_GROUP_SIZE                    | 4096
CL_KERNEL_COMPILE_WORK_GROUP_SIZE            | (0, 0, 0)
CL_KERNEL_LOCAL_MEM_SIZE                     | 0 bytes
CL_KERNEL_PRIVATE_MEM_SIZE                   | 0 bytes
CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE | 8
Send command from host to device   | 1.172 μs
Including data transfer            | 37.303 μs
Execution of kernel                | 661.000 ns
```

mypi (generic function with 1 method)

```
function mypi(; niters = 10, in_nsteps = 10)
    cl.device!(π_device)

    prg = cl.Program(; source = π_code.code) |> cl.build!
    pi_kernel = cl.Kernel(prg, "pi")
    work_group_size = cl.device().max_work_group_size
    nwork_groups = in_nsteps ÷ (work_group_size * niters)
    nsteps = work_group_size * niters * nwork_groups

    step_size = 1.0 / nsteps

    global_size = (nwork_groups * work_group_size,)
    local_size  = (work_group_size,)
    localmem    = cl.LocalMem(Float32, work_group_size)

    h_psum = Vector{Float32}(undef, nwork_groups)
    d_partial_sums = CLArray{Float32}(undef, length(h_psum))
    timed_clcall(pi_kernel, Tuple{Int32, Float32, cl.LocalMem{Float32}, Ptr{Float32}},
       niters, step_size, localmem, d_partial_sums; global_size, local_size)
    cl.copy!(h_psum, d_partial_sums)

    return sum(h_psum) * step_size
end
```

▶ **How to compute π with a kernel ?**

Portable Computing Language

cpu-haswell-AMD EPYC 7763 64-Core Processor

Sum

```
vec =
▸[0.33825, 0.633399, 0.178376, 0.205412, 0.537014, 0.851464, 0.00113535, 0.362713, 0.343
```

```julia
vec = rand(Float32, local_len)
```

```
6.7207074f0
```

```julia
@btime sum(vec)
```

```
  29.311 ns (1 allocation: 16 bytes)
```

```
6.720708f0
```

```julia
local_sum(vec)
```

```
CL_KERNEL_WORK_GROUP_SIZE                    | 4096
CL_KERNEL_COMPILE_WORK_GROUP_SIZE            | (0, 0, 0)
CL_KERNEL_LOCAL_MEM_SIZE                     | 0 bytes
CL_KERNEL_PRIVATE_MEM_SIZE                   | 0 bytes
CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE | 8
Send command from host to device   | 1.202 μs
Including data transfer            | 57.787 ms
Execution of kernel                | 20.297 μs
```

```
local_sum (generic function with 1 method)
```

```julia
function local_sum(x::Vector{T}) where {T}
    cl.device!(local_device)
    global_x = CLArray(x)
    local_x = cl.LocalMem(T, length(global_x))
    result = CLArray(zeros(T, 1))

    prg = cl.Program(; source = local_sum_code.code) |> cl.build!
    k = cl.Kernel(prg, "local_sum")

    timed_clcall(k, Tuple{Ptr{T}, cl.LocalMem{T}, Ptr{T}}, global_x, local_x, result; global_size=length(global_x))

    return Array(result)[]
end
```

How to compute the sum an array in local memory with a kernel ?

Portable Computing Language

cpu-haswell-AMD EPYC 7763 64-Core Processor

local_len = 16

Utils

```
timed_clcall (generic function with 1 method)
```

```julia
function timed_clcall(kernel, args...; kws...)
    info = cl.work_group_info(kernel, cl.device())
    # See https://registry.khronos.org/OpenCL/sdk/3.0/docs/man/html/clGetKernelWorkGroupInfo.html
    println("CL_KERNEL_WORK_GROUP_SIZE                    | ", info.size)
    println("CL_KERNEL_COMPILE_WORK_GROUP_SIZE            | ", info.compile_size)
    println("CL_KERNEL_LOCAL_MEM_SIZE                     | ", BenchmarkTools.prettymemory(info.local_mem_size))
    println("CL_KERNEL_PRIVATE_MEM_SIZE                   | ", BenchmarkTools.prettymemory(info.private_mem_size))
    println("CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE | ", info.prefered_size_multiple)

    # ':profile' sets `CL_QUEUE_PROFILING_ENABLE` to the command queue
    cl.queue!(:profile) do
        evt = clcall(kernel, args...; kws...)
        wait(evt)

        # See https://registry.khronos.org/OpenCL/sdk/3.0/docs/man/html/clGetEventProfilingInfo.html
        t = evt.profile_duration
        println("Send command from host to device  | $(BenchmarkTools.prettytime(evt.profile_submit - evt.profile_queued))")
        println("Including data transfer           | $(BenchmarkTools.prettytime(evt.profile_start - evt.profile_submit))")
        println("Execution of kernel               | $(BenchmarkTools.prettytime(evt.profile_end - evt.profile_start))")
    end
end
```

```
Activating project at `~/work/LINMA2710/LINMA2710/Lectures`
```

```julia
using OpenCL, pocl_jll # `pocl_jll` provides the POCL OpenCL platform for CPU devices
```